ArenaHub

Guia do sistema

Backend, frontend e banco: o que cada peca faz, por que ela existe, e por onde passa cada uma das 25 funcionalidades.

6	7	37	6	20	8
Models	Controllers	Services	Repositories	Procedures	Telas

Flask + Flask-SQLAlchemy + SQLite · github.com/paulim21/hub_arena_ps

1. O caminho de uma requisicao

Toda funcionalidade do sistema faz o mesmo percurso. O exemplo abaixo e o de criar uma reserva, que e o mais completo: ele passa por todas as camadas e ainda consulta uma procedure no meio do caminho.

1. TELA `frontend/reservas.html`	Monta o JSON e chama fetch('/api/reservas', POST). Não calcula valor nenhum.
2. ROTA `backend/routes.py`	Mapa de URL para metodo de Controller. 34 rotas, nenhuma logica.
3. CONTROLLER `ReservaController.criar`	Le o corpo JSON, chama o Service, traduz excecao em status HTTP. Sem regra.
4. SERVICE `CriarReservaService.executar`	Valida, pergunta ao Repository se ha conflito, calcula horas x preco.
5. MODEL / REPOSITORY `Reserva.salvar / ReservaRepository`	Model persiste; Repository executa a procedure sp_conflito_reserva.
6. BANCO `backend/hub_arena.db`	SQLite com as 6 tabelas e a tabela procedures.

A pergunta que a banca faz: por que tantas camadas?

Porque cada uma responde por uma coisa so, e isso aparece quando algo muda. Trocar SQLite por MySQL mexe no Repository e nao toca no Service. Mudar a regra de cobranca mexe no Service e nao toca na Controller. Trocar a tela nao mexe em nada do backend, porque a tela so fala com a API. Em um arquivo unico, qualquer uma dessas mudancas obriga a reler tudo.

O valor da reserva e calculado no SERVIDOR, nunca na tela

O frontend manda quadra, inicio, fim e jogadores. Quem multiplica horas pelo preco da quadra e o CriarReservaService. Se o valor viesse da tela, bastaria abrir o console do navegador e mandar 'valor_total: 0'.

2. Mapa do backend, arquivo por arquivo

Arquivo ou pasta	Do que responde
`app.py`	Application Factory: cria a aplicacao, liga o SQLAlchemy, ativa as chaves estrangeiras do SQLite, cria as tabelas e registra as rotas.
`routes.py`	Mapa de URL -> metodo de Controller. Os Blueprints vivem aqui, e nenhuma logica mora neste arquivo.
`extensions.py`	O objeto db, criado sem vinculo com a app para que Models e Repositories possam importar sem import circular.
`database.py`	Registra as procedures no banco e aplica as migracoes de coluna (create_all nao altera tabela que ja existe).
`politica.py`	A versao e as finalidades dos termos de uso, num lugar so.
`validadores.py`	A forma de CPF, CNPJ, telefone e e-mail: digito verificador, DDD e normalizacao.
`models/`	6 classes que herdam de db.Model. Cada uma e uma tabela, e carrega o CRUD basico.
`controllers/`	6 classes, uma por recurso. Recebem HTTP e devolvem HTTP.
`services/`	37 classes, uma por caso de uso. Toda a regra de negocio.
`repositories/`	6 classes para consultas complexas, mais o runner que executa as procedures e o arquivo procedures.sql.

3. Models: as entidades do dominio

Herdam de `db.Model`, e e essa heranca que mapeia cada classe a uma tabela. O CRUD basico fica na propria Model, como pede a arquitetura da disciplina: `salvar()`, `atualizar()`, `deletar()`, `listar_todos()` e `buscar_por_id()`.

Model	Tabela	Colunas	O que representa
Arena	`arenas`	8	Pertence a um dono (FK para usuarios) e agrega quadras.
Consentimento	`consentimentos`	8	O aceite dos termos, com versao, data, IP e navegador. Nao tem FK para Usuario de proposito (ver a secao do consentimento LGPD).
Quadra	`quadras`	5	Pertence a uma arena. O preco por hora daqui e a base do valor da reserva.
Reserva	`reservas`	7	Quadra, periodo, valor e status. salvar() e redefinida para gravar a reserva e os jogadores na mesma transacao.
ReservaJogador	`reserva_jogadores`	2	Classe associativa que resolve o N:N entre Reserva e Usuario. Chave primaria composta, e ON DELETE CASCADE.
Usuario	`usuarios`	8	Atleta, dono ou professor. E-mail e CPF sao UNIQUE; a senha e guardada como resumo scrypt, nunca em texto.

Relacionamentos, na linguagem do diagrama de classes

Composicao (a parte nao existe sem o todo): Arena para Quadra, e Reserva para ReservaJogador. **Agregacao** (a parte existe por si so): Usuario para Arena, Quadra para Reserva, Usuario para Reserva e Usuario para ReservaJogador. O diagrama completo esta em docs/diagrama-classes.svg.

4. Como o SQLAlchemy funciona aqui

SQLAlchemy e o ORM do projeto: a camada que faz uma classe Python virar uma tabela e um objeto virar uma linha. Flask-SQLAlchemy nao e outro ORM, e so a cola que liga esse ORM ao ciclo de vida da aplicacao Flask.

As tres pecas

Peca	No projeto	Do que responde
Engine	criado em app.py	Quem de fato fala com o SQLite: abre a conexao, manda o SQL, traz o resultado. Nasce uma vez, a partir de SQLALCHEMY_DATABASE_URI.
Session	db.session	O rascunho da transacao. Guarda tudo o que foi criado, alterado ou apagado e so escreve no banco quando vem o commit.
Model	models/*.py	A classe mapeada. Cada db.Column e uma coluna, e e a heranca de db.Model que registra o mapeamento.

De classe para tabela

Nao existe SQL de criacao escrito a mao em nenhum lugar do projeto. O que existe e a classe; o CREATE TABLE e consequencia dela.

```
class Usuario(db.Model):
    __tablename__ = 'usuarios'
    usuario_id = db.Column(db.Integer, primary_key=True)
    email = db.Column(db.String(120), unique=True, nullable=False)

-- o que o SQLAlchemy gera
CREATE TABLE usuarios (
    usuario_id INTEGER NOT NULL PRIMARY KEY,
    email VARCHAR(120) NOT NULL UNIQUE
);
```

create_all CRIA tabela que falta, mas nunca ALTERA tabela que existe

Isso virou erro de verdade no projeto. Ao acrescentar a coluna cpf em Usuario, o create_all viu que a tabela usuarios ja existia e nao fez nada: a tela passou a responder `no such column: usuarios.cpf`. A correcao foi `migrar_colunas_novas()`, em database.py: ele le o PRAGMA table_info de cada tabela e emite ALTER TABLE ADD COLUMN para o que faltar. ORM nao dispensa migracao.

add, flush, commit e rollback: quatro momentos diferentes

Chamada	O que acontece no banco	Onde o projeto usa
add	Nada ainda. O objeto fica pendurado na Session, e a chave primaria continua None.	Dentro de salvar(), em toda Model.
flush	Manda o INSERT e NAO fecha a transacao. O banco devolve o id autoincremento.	Reserva.salvar(): precisa do reserva_id para inserir os jogadores.
commit	Confirma tudo de uma vez. Se qualquer parte falhar, nada fica.	Fim de salvar(), atualizar() e deletar().
rollback	Desfaz a transacao inteira e limpa a Session.	Obrigatorio depois de IntegrityError: sem ele a Session fica suja e a proxima chamada falha sem motivo aparente.

Por que atualizar() e so um commit()

A Session vigia os objetos que ela carregou. No momento em que o Service escreve `arena.nome = 'Novo nome'`, o objeto ja fica marcado como sujo. Quando o commit chega, o SQLAlchemy compara o estado atual com o que leu do banco e escreve o UPDATE apenas das colunas que mudaram. Por isso atualizar() nao tem corpo nenhum: nao ha UPDATE para escrever a mao, e nao ha coluna para esquecer.

O mapa de identidade. Dentro da mesma Session, pedir o mesmo registro duas vezes devolve o MESMO objeto em memoria, nao duas copias. E por isso que AtualizarArenaService pode chamar BuscarArenaService sem medo: a segunda busca nao custa outra consulta, e nao existe risco de duas versoes do mesmo registro discordando dentro da mesma requisicao.

relationship e a armadilha do N+1. O relationship entrega `arena.quadras` sem escrever JOIN. Comodo, e caro: um laco por 50 arenas pedindo .quadras faz 51 consultas. E exatamente por isso que a listagem com dono e contagem de quadras nao e um laco em Python, e sim uma procedure com JOIN e COUNT na camada Repository.

Onde o projeto sai do ORM de proposito. Relatorio, busca e disponibilidade sao SQL escrito em repositories/procedures.sql e executado com `db.session.execute(text(...))`. O ORM e bom para uma entidade por vez; um GROUP BY sobre tres tabelas e mais claro em SQL. Essa escolha e justamente a fronteira entre Model e Repository.

O SQLite ignora chave estrangeira se ninguem pedir

Por compatibilidade historica, o SQLite so respeita FK quando a conexao executa `PRAGMA foreign_keys = ON` — e o pragma vale por conexao, nao por banco. O app.py escuta o evento connect do Engine e roda o pragma em toda conexao nova. Sem isso, o ON DELETE CASCADE dos jogadores nao aconteceria e as exclusoes bloqueadas passariam direto: as regras de integridade seriam enfeite.

5. Controllers: a fronteira com o HTTP

Sao classes, uma por recurso. Fazem tres coisas e nada mais: leem o que veio na requisicao, chamam o Service correspondente e traduzem o resultado (ou a excecao) em resposta HTTP com o status certo.

Controller	Metodos	Recurso
ArenaController	6	`/api/arenas`
BuscaController	1	`/api/buscas`
ConsentimentoController	5	`/api/consentimentos`
QuadraController	6	`/api/quadras`
RelatorioController	5	`/api/relatorios`
ReservaController	5	`/api/reservas`
UsuarioController	6	`/api/usuarios`

A traducao de excecao em status e a convencao do projeto:

Excecao no Service	Status HTTP	Quando acontece
ValueError	400	Dado invalido ou regra de negocio violada (CPF errado, horario em conflito)
LookupError	404	O registro pedido nao existe
IntegrityError	400	O banco recusou (UNIQUE ou chave estrangeira), e o Service traduz em mensagem legivel

O que NAO pode aparecer numa Controller

Calculo, validacao de regra, SQL e chamada de procedure. Se a banca abrir um arquivo de controller e encontrar um 'if' decidindo preco, a separacao de camadas caiu.

6. Services: um por caso de uso

Cada classe responde por **uma acao** do sistema e expoe um unico metodo `executar()`. E aqui que mora toda a regra de negocio, e e por isso que sao 37 classes e nao seis.

```
AgendaArenaService                CriarQuadraService                OcupacaoQuadrasService
AtualizarArenaService             CriarReservaService               PesquisarUsuariosService
AtualizarQuadraService            CriarUsuarioService               QuadrasDisponiveisService
AtualizarStatusReservaService     DeletarArenaService               RankingAtletasService
AtualizarUsuarioService           DeletarQuadraService              ReceitaPorArenaService
BuscaGlobalService                DeletarReservaService             RegistrarConsentimentoService
BuscarArenaService                DeletarUsuarioService             RelatorioConsentimentosService
BuscarQuadraService               ListarArenasComDonoService        ReservasRecentesService
BuscarReservaService              ListarArenasService               ResumoDashboardService
BuscarUsuarioService              ListarQuadrasService              RevogarConsentimentoService
ConferirSenhaService              ListarReservasDetalhadasService   VerificarConsentimentoService
ConsultarPoliticaService          ListarReservasService
CriarArenaService                 ListarUsuariosService
```

Os tres Services que vale abrir na apresentacao

Service	Por que ele e interessante
CriarReservaService	Concentra as duas regras mais citadas do projeto: pergunta ao Repository se ha conflito de horario (anti-overbooking) e calcula o valor como horas x preco da quadra. Tambem aceita varios jogadores, gravando o N:N.
CriarUsuarioService	Valida a forma dos dados antes de tocar o banco (CPF com digito verificador, e-mail normalizado, telefone com DDD) e transforma a senha em resumo scrypt. Quando o UNIQUE estoura, diz QUAL campo colidiu.
RegistrarConsentimentoService	Recusa consentimento parcial e versao desatualizada dos termos, e revoga o aceite anterior do mesmo aparelho antes de gravar o novo.

7. Repository e as procedures

O Repository nao repete CRUD simples: isso ja esta na Model. Ele existe para **consulta complexa** — JOIN, GROUP BY, NOT EXISTS, ranking, relatorio — e e o unico lugar do sistema que executa SQL.

Como existem procedures em SQLite

O SQLite e um banco embarcado e nao tem CREATE PROCEDURE, que e recurso de SGBD cliente-servidor. Para atender ao requisito, cada procedure e **armazenada na propria tabela procedures do banco** (nome + SQL parametrizado), registrada na subida do servidor a partir de repositories/procedures.sql. Quem faz o papel do CALL e o procedure_runner.py, dentro do Repository. Controller e Service nunca executam SQL.

As 20 procedures

Procedure	Para que serve
`sp_usuarios_por_tipo`	Filtro de usuarios por perfil
`sp_busca_usuarios`	Busca textual por nome ou e-mail (LIKE)
`sp_arenas_por_cidade`	Filtro de arenas por cidade
`sp_arenas_com_dono`	JOIN com o dono + contagem de quadras
`sp_quadras_com_arena`	JOIN quadra/arena
`sp_quadras_por_arena`	Quadras de uma arena
`sp_conflito_reserva`	ANTI-OVERBOOKING: reservas que colidem no periodo
`sp_reservas_detalhadas`	JOIN triplo reserva/quadra/arena
`sp_reservas_recentes`	Ultimas reservas do painel
`sp_jogadores_da_reserva`	Nomes dos jogadores (JOIN no N:N)
`sp_resumo_dashboard`	Contadores e receita total
`sp_receita_por_arena`	RELATORIO: JOIN + GROUP BY + ticket medio
`sp_ocupacao_quadras`	RELATORIO: horas reservadas e receita por quadra
`sp_ranking_atletas`	RELATORIO: ranking por jogos e horas
`sp_quadras_disponiveis`	BUSCA: quadras livres no periodo (NOT EXISTS)
`sp_agenda_arena`	Agenda da arena no periodo
`sp_consentimento_vigente`	LGPD: o aceite ainda valido deste aparelho
`sp_consentimentos_por_versao`	LGPD: aceites, vigentes e revogados por versao
`sp_busca_arenas`	
`sp_busca_quadras`	

8. O banco de dados

SQLite, num arquivo unico: `backend/hub_arena.db`. Ele e criado sozinho na primeira execucao e nao vai para o repositorio — esta no .gitignore, porque banco com dado de gente dentro nao se versiona.

Tabela	Colunas	Chaves
usuarios	usuario_id, nome, email, cpf, senha, telefone, tipo_usuario, data_criacao	PK usuario_id; UNIQUE em email e cpf
arenas	arena_id, nome, cnpj, endereco, cidade, dono_id, taxa_saas, dia_vencimento	PK arena_id; FK dono_id -> usuarios
quadras	quadra_id, arena_id, nome, tipo_esporte, preco_hora	PK quadra_id; FK arena_id -> arenas
reservas	reserva_id, quadra_id, usuario_reserva_id, data_hora_inicio, data_hora_fim, valor_total, status	PK reserva_id; FK para quadras e usuarios
reserva_jogadores	reserva_id, usuario_id	PK composta; FK para reservas (ON DELETE CASCADE) e usuarios
consentimentos	consentimento_id, identificador, versao_termos, finalidades, aceito_em, revogado_em, origem_ip, agente	PK consentimento_id; sem FK, de proposito
procedures	nome, sql	PK nome. Guarda as 20 procedures do sistema

A chave estrangeira do SQLite so vale se for ligada

O SQLite ignora FK por padrao, e so respeita se o PRAGMA foreign_keys estiver ON **em cada conexao**. O app.py liga isso num evento de conexao do SQLAlchemy. E por causa dele que excluir um dono com arena, uma arena com quadras ou uma quadra com reservas e recusado pelo banco — e o Service traduz a recusa numa frase legivel.

Por que as datas sao TEXT e nao DATETIME

O SQLite nao tem tipo de data: ele guarda texto, numero ou blob. As datas ficam em texto no formato ISO (2026-12-05T19:00) porque esse formato **ordena cronologicamente quando comparado como texto**. E isso que faz o BETWEEN dos relatorios e a comparacao do anti-overbooking funcionarem sem conversao.

9. As 25 funcionalidades, de ponta a ponta

Cada linha e um caminho completo: tela, rota, Controller, Service e, quando ha consulta complexa, a procedure que o Repository executa.

#	Funcionalidade	Rota	Service	Procedure
1	Cadastrar usuario	`POST /api/usuarios`	`CriarUsuarioService`	-
2	Listar/filtrar por tipo	`GET /api/usuarios?tipo=`	`ListarUsuariosService`	`sp_usuarios_por_tipo`
3	Buscar por nome ou e-mail	`GET /api/usuarios/busca`	`PesquisarUsuariosService`	`sp_busca_usuarios`
4	Excluir usuario	`DELETE /api/usuarios/`	`DeletarUsuarioService`	-
5	Cadastrar arena	`POST /api/arenas`	`CriarArenaService`	-
6	Listar arenas com dono	`GET /api/arenas?detalhes=1`	`ListarArenasComDonoService`	`sp_arenas_com_dono`
7	Excluir arena	`DELETE /api/arenas/`	`DeletarArenaService`	-
8	Cadastrar quadra	`POST /api/quadras`	`CriarQuadraService`	-
9	Listar/filtrar quadras	`GET /api/quadras`	`ListarQuadrasService`	`sp_quadras_com_arena`
10	Excluir quadra	`DELETE /api/quadras/`	`DeletarQuadraService`	-
11	Criar reserva	`POST /api/reservas`	`CriarReservaService`	`sp_conflito_reserva`
12	Listar reservas detalhadas	`GET /api/reservas?detalhes=1`	`ListarReservasDetalhadasService`	`sp_reservas_detalhadas`
13	Confirmar reserva	`PUT /api/reservas/`	`AtualizarStatusReservaService`	-
14	Cancelar reserva	`DELETE /api/reservas/`	`DeletarReservaService`	-
15	Painel com resumo	`GET /api/relatorios/resumo`	`ResumoDashboardService`	`sp_resumo_dashboard`
16	Receita por arena	`GET /api/relatorios/receita-arenas`	`ReceitaPorArenaService`	`sp_receita_por_arena`
17	Ocupacao das quadras	`GET /api/relatorios/ocupacao-quadras`	`OcupacaoQuadrasService`	`sp_ocupacao_quadras`
18	Ranking de atletas	`GET /api/relatorios/ranking-atletas`	`RankingAtletasService`	`sp_ranking_atletas`
19	Quadras disponiveis	`GET /api/quadras/disponiveis`	`QuadrasDisponiveisService`	`sp_quadras_disponiveis`
20	Agenda da arena	`GET /api/arenas//agenda`	`AgendaArenaService`	`sp_agenda_arena`
21	Registrar consentimento	`POST /api/consentimentos`	`RegistrarConsentimentoService`	-
22	Verificar consentimento	`GET /api/consentimentos/vigente`	`VerificarConsentimentoService`	`sp_consentimento_vigente`
23	Revogar consentimento	`POST /api/consentimentos/revogar`	`RevogarConsentimentoService`	-
24	Relatorio de consentimentos	`GET /api/consentimentos/relatorio`	`RelatorioConsentimentosService`	`sp_consentimentos_por_versao`
25	Busca global	`GET /api/busca`	`BuscaGlobalService`	`sp_busca_usuarios` + `sp_busca_arenas` + `sp_busca_quadras`

10. O que nao e CRUD

A disciplina pede CRUD, e ele esta la. Mas vale dizer na apresentacao que a maior parte do sistema nao e CRUD: de 25 funcionalidades, 15 nao se resumem a inserir, ler, atualizar ou apagar uma linha.

Grupo	Quantas	Quais	Por que estao nesse grupo
Nao e CRUD	15	3, 6, 11, 12, 15 a 25	Agregacao, juncao, busca em tres tabelas, calculo ou decisao. A resposta nao e uma linha de tabela.
CRUD puro	8	1, 4, 5, 7, 8, 10, 13, 14	Uma operacao sobre uma entidade so: inserir, atualizar status ou apagar.
Leitura com filtro	2	2, 9	A mesma leitura, mudando o WHERE.

As 15, e o que cada uma faz que CRUD nao faz

#	Funcionalidade	O que faz que CRUD nao faz
3	Buscar usuario por nome ou e-mail	LIKE em duas colunas ao mesmo tempo, com o termo normalizado antes.
6	Arenas com dono e contagem	JOIN com usuarios mais COUNT de quadras: a linha devolvida nao existe em tabela alguma.
11	Criar reserva	Pergunta a uma procedure se ha sobreposicao de horario, calcula horas x preco e grava duas tabelas na mesma transacao.
12	Reservas detalhadas	JOIN triplo (reserva, quadra, arena) e os nomes dos jogadores agregados em um campo.
15	Resumo do painel	COUNT e SUM de tabelas diferentes reunidos em uma resposta so.
16	Receita por arena	SUM agrupado por arena, contando apenas reserva confirmada.
17	Ocupacao das quadras	Horas reservadas sobre horas disponiveis: um percentual que nao esta guardado em nenhum lugar.
18	Ranking de atletas	COUNT por jogador na tabela associativa, com ORDER BY decrescente.
19	Quadras disponiveis	NOT EXISTS: devolve o que NAO esta na outra tabela naquele periodo. O contrario de uma leitura.
20	Agenda da arena	Recorte por periodo com ordenacao cronologica, validando as datas ISO antes.
21	Registrar consentimento	Recusa aceite parcial e aceite de versao que nao e a vigente.
22	Consentimento vigente	Decide se vale: confere a versao e se existe revogacao depois do aceite.
23	Revogar consentimento	Marca a data de revogacao e NAO apaga o registro, porque ele e a prova.
24	Relatorio de consentimentos	Agrupa por versao dos termos e conta aceitos e revogados de cada uma.
25	Busca global	Um termo, tres repositorios, tres procedures e uma resposta unificada.

E os 8 de CRUD puro tambem tem regra

CRUD aqui e o formato da operacao, nao a ausencia de regra. Cadastrar usuario confere digito verificador de CPF e guarda a senha como resumo scrypt. Excluir arena apanha o erro de integridade do banco e devolve uma frase que diz o que apagar primeiro. Confirmar reserva so aceita status que esta na lista de validos. Nenhuma dessas decisoes esta na Controller nem na Model: todas estao no Service.

11. O frontend

HTML, CSS e JavaScript puro — sem framework e sem Jinja2. O servidor entrega arquivos estaticos, e cada tela busca os proprios dados chamando a API por fetch. Nenhuma pagina e montada no servidor.

As 8 telas

Arquivo	Rota	O que faz
`arenas.html`	`/arenas`	Lista com dono e numero de quadras, cadastro e exclusao
`busca.html`	`/busca`	Busca global: um termo em usuarios, arenas e quadras
`index.html`	`/`	Visao geral: contadores animados, receita e atividade recente
`quadras.html`	`/quadras`	Lista com filtro por arena, cadastro e exclusao
`relatorios.html`	`/relatorios`	Receita, ocupacao, ranking, disponibilidade, agenda e consentimentos
`reservas.html`	`/reservas`	Agenda, criacao com anti-overbooking, confirmar e cancelar
`termos.html`	`/termos`	Politica de privacidade completa, com revogacao do consentimento
`usuarios.html`	`/usuarios`	Lista, filtro por tipo, cadastro e exclusao

Os 4 arquivos de JavaScript, e a ordem em que carregam

A ordem importa, e nao e detalhe: cada um envolve o anterior.

Arquivo	Quando carrega	Do que responde
tema.js	No , SEM defer	Escreve o tema no HTML antes da primeira pintura. Com defer, a tela piscaria clara antes de escurecer, na cara de quem ja esta lendo.
base.js	Antes do script da pagina	HubUI.api (o fetch), os icones, os , os menus suspensos, o seletor de data, as mascaras de CPF/CNPJ/telefone e os formatos.
interface.js	Depois do base.js	Envolve o HubUI.api para acender a barra de progresso, e traz o esqueleto de carregamento.
lgpd.js	Por ultimo	Envolve o HubUI.api de novo, por fora, para SEGURAR toda chamada ate existir consentimento. Por fora da barra, ela nao acende enquanto o aviso esta na tela: nao ha nada carregando, ha alguem lendo.

O frontend nao decide nada que valha dinheiro

Ele nao calcula valor de reserva, nao decide se um horario esta livre e nao valida CPF sozinho. Tudo isso o servidor refaz. O que a tela faz e cuidar da experiencia: mascara enquanto se digita, esqueleto no lugar do vazio, erro que espera ser lido.

11.1 O sistema de design

A aparencia tambem tem regra escrita, e ela e verificada por maquina.

Arquivo	Para que serve
frontend/css/tokens.css	TODOS os valores: cor, tamanho de texto, espacamento, raio, borda e duracao. Fora dele nao existe valor literal.
frontend/css/estilo.css	So componentes. Nenhuma cor escrita a mao.
IDENTIDADE_VISUAL.md	O porque de cada decisao, com as tabelas de contraste medidas pela formula da WCAG 2.1. Os numeros sao contrato.
docs/design/PADROES.md	O como: o que ja existe, o que nao fazer, e as armadilhas que ja custaram correcao.
docs/design/verificar-identidade.py	Mede o contrato: contraste de cada par, distancia de matiz entre sinal e marca, cor literal fora do tokens.css, escala de texto, grade de 4px, gradiente, emoji e a contagem de icones. Imprime quantas verificacoes fez ao terminar.

Duas decisoes que aparecem em toda tela

Nenhum emoji na interface. Emoji e desenhado pela fonte do sistema: muda de forma entre aparelhos, nao herda cor e nao aceita o peso do traco da marca. No lugar vao 43 icones em SVG, desenhados no mesmo peso.

Navy escreve, areia e chao. A cor da marca vem da logo. O azul escreve (link, aba ativa, acao principal) e a areia e fundo (faixa, item escolhido). Trocar os papeis quebra o contraste: a areia tem 1,25:1 contra o papel, e sumiria como texto.

12. Os botoes: de onde vem cada um

Nenhum botao do sistema e uma tag solta com onclick escrito no HTML. Eles vem de quatro lugares, e saber de qual deles encurta muito a procura no codigo.

Origem	Exemplos	Onde esta	Como o clique e ligado
Escrito no HTML da tela	Cadastrar, Buscar, Novo	frontend/<tela>.html	O script da propria tela faz addEventListener no carregamento.
Gerado pelo JS ao desenhar a lista	Confirmar, Cancelar, Excluir de cada linha	frontend/js/<tela>.js	Nascem junto com a linha e carregam o id no dataset; um listener no container atende todos.
Construido pelo base.js	Menu suspenso, seletor de data, multi-selecao, dialogos de confirmar e avisar, botao de tema	frontend/js/base.js	O base.js varre a tela procurando classes (.dd, .dtp, .multi-sel) e substitui o controle nativo pelo proprio.
Do aviso de LGPD	Aceito os termos, Ler os termos, Revogar	frontend/js/lgpd.js e termos.html	O lgpd.js monta o dialogo na primeira visita e liga os botoes antes de liberar qualquer chamada.

A regra visual: um preenchido por tela

Todo botao e de traco por padrao — contorno e texto em navy, fundo transparente. Apenas UM por tela recebe a classe principal, preenchido de navy: a acao pela qual aquela tela existe (Cadastrar em usuarios, Reservar em reservas). Dois preenchidos na mesma tela ja e erro de hierarquia, porque nenhum dos dois ganha o olho de quem chega. Acao destrutiva nunca e preenchida: Excluir e traco com o vermelho de sinal.

O que acontece do lado da tela, em ordem

Passo	Quem	O que faz
1	o listener da tela	Le os campos, monta o objeto e chama HubUI.api(metodo, rota, corpo).
2	lgpd.js	Envolve o HubUI.api por FORA. Sem consentimento vigente, a chamada nem sai: ela fica esperando o aceite.
3	interface.js	Envolve o HubUI.api por dentro e acende a barra de progresso no topo — depois de 120 ms, para que clique rapido nao pisque a tela.
4	HubUI.api	Faz o fetch, le o JSON e levanta erro com a mensagem do servidor quando o status nao e de sucesso.
5	o listener de novo	Deu certo: redesenha a lista e mostra um toast. Deu erro: escreve a mensagem do servidor no topo do formulario, legivel, sem alert().

13. O backend depois do clique

Do outro lado, todo clique percorre os mesmos seis passos. O que muda de uma funcionalidade para outra e o Service; a estrutura nao muda nunca.

Passo	Quem	O que faz
1	routes.py	Casa URL mais metodo HTTP com um metodo de Controller, via add_url_rule. Nenhuma logica mora aqui.
2	Controller	Le request.get_json() ou os parametros da query e chama o Service. Nao decide nada.
3	Service	Toda a regra: valida, confere o que precisa existir, consulta, calcula e decide. Levanta excecao quando a regra nao fecha.
4	Model ou Repository	Model persiste uma entidade pelo ORM; Repository executa procedure quando a consulta e complexa.
5	SQLite	A transacao e confirmada inteira — ou desfeita inteira.
6	Controller	Transforma o retorno em JSON com o status HTTP certo. A tabela de excecao para status esta na secao das Controllers.

Botao por botao, o que o backend faz

Botao	Rota	O que o backend faz, na ordem
Cadastrar usuario	`POST /api/usuarios`	CriarUsuarioService normaliza CPF, telefone e e-mail, confere os digitos verificadores, gera o resumo scrypt da senha e tenta inserir. Se o UNIQUE colidir, ele descobre se foi o e-mail ou o CPF e devolve a frase certa, nao um erro generico.
Reservar horario	`POST /api/reservas`	CriarReservaService tira jogadores repetidos, confere se a quadra e cada jogador existem, le as datas ISO, pergunta a sp_conflito_reserva se ha sobreposicao, calcula horas x preco_hora, grava a reserva com flush() para obter o id e insere os jogadores — tudo em um commit() so.
Confirmar	`PUT /api/reservas/<id>`	AtualizarStatusReservaService aceita apenas pendente ou confirmada. Ele muda o atributo do objeto; quem escreve o UPDATE e o commit, comparando o que mudou.
Cancelar	`DELETE /api/reservas/<id>`	DeletarReservaService apaga a reserva. Os jogadores vao com ela por ON DELETE CASCADE, sem nenhuma linha de Python para isso.
Excluir arena, quadra ou usuario	`DELETE /api/...`	O Service tenta apagar, o SQLite recusa por chave estrangeira, ele faz rollback() e levanta uma mensagem dizendo o que apagar primeiro. Vira 409, nao 500: nao foi falha do sistema, foi a regra funcionando.
Buscar	`GET /api/busca?termo=`	BuscaGlobalService exige 2 caracteres, chama tres procedures (usuarios, arenas e quadras) e devolve as tres listas juntas. CPF nao e pesquisavel de proposito.
Abrir um relatorio	`GET /api/relatorios/...`	O Service valida os parametros e o Repository executa a procedure. A conta e feita pelo SQLite, nao em Python: e menos dado atravessando a rede e menos lugar para errar.
Aceito os termos	`POST /api/consentimentos`	RegistrarConsentimentoService confere se a versao enviada e a vigente, recusa aceite parcial e grava versao, data, IP e navegador — a prova que a lei pede.

O que o aviso de LGPD NAO faz — e vale dizer antes de perguntarem

O bloqueio vive no navegador: ele segura as telas, nao a API. Quem chamar /api/usuarios por fora, com curl ou Postman, e atendido sem passar pelo aceite. Para um trabalho de disciplina isso e o esperado, porque o sistema nao tem login. Num sistema em producao a verificacao teria de ficar no servidor, amarrada a uma sessao autenticada. Dizer isso na apresentacao vale mais do que torcer para ninguem perguntar.

14. O consentimento LGPD

O sistema nao abre antes do aceite: o frontend segura toda chamada a API ate existir consentimento. Mas a parte que importa para a defesa esta no backend.

O que a lei pede	Onde isso esta no codigo
Comprovar o consentimento obtido (Art. 8, par. 1)	Cada aceite e uma linha na tabela consentimentos, com versao dos termos, data, IP e navegador. Um sinalizador no navegador nao comprova nada.
Consentimento por finalidade; o generico e nulo (Art. 8, par. 4)	politica.py declara as finalidades separadas, e o RegistrarConsentimentoService recusa aceite parcial.
Novo aceite quando a finalidade muda (Art. 8, par. 6)	O campo versao_termos. O Service recusa aceite de versao que nao e a vigente; subir a VERSAO faz todo mundo aceitar de novo.
Revogacao livre e gratuita (Art. 8, par. 5)	RevogarConsentimentoService preenche revogado_em. O registro NAO e apagado: ele e a prova, inclusive da revogacao.
Medidas de seguranca (Art. 46)	A senha vai ao banco como resumo scrypt com sal. Nem a equipe consegue ler a senha de um usuario.
Minimizacao (Art. 6, III)	Consentimento guarda um identificador ALEATORIO do navegador, e nao o usuario_id. Por isso a Model nao tem chave estrangeira para Usuario.

15. Validacao e seguranca

O arquivo `validadores.py` conhece a forma dos dados que tem forma. Ele fica fora dos Services de proposito: nao e caso de uso, e a forma do dado, e ela e a mesma para quem cadastra e para quem atualiza.

Dado	O que e conferido	Como vai ao banco
CPF	11 digitos, digitos verificadores calculados, e recusa de sequencias repetidas como 111.111.111-11	12345678909
CNPJ	14 digitos e digitos verificadores (opcional na arena)	11222333000181
Telefone	DDD valido e 8 ou 9 digitos; celular de 9 precisa comecar com 9; o 55 do pais e descartado	31999990000
E-mail	formato, e normalizacao para minusculas (senao Ana@x.com e ana@x.com viram duas contas)	ana@email.com
Senha	minimo de 6 caracteres	resumo scrypt com sal

A pontuacao e apresentacao, e apresentacao nao se persiste

O banco guarda 12345678909, nunca 123.456.789-09. Gravar formatado obriga todo filtro, toda busca e todo indice a adivinhar qual das grafias veio. A mascara aparece ao digitar, no frontend, e a formatacao para leitura acontece na tela.

Integridade no banco

O SQLite so respeita chave estrangeira se o PRAGMA for ligado **em cada conexao**, e e isso que o app.py faz. E por causa disso que excluir um dono com arena, uma arena com quadras ou uma quadra com reservas e recusado — e o Service traduz esse erro do banco em uma frase que a pessoa entende.

16. Como rodar e o que mostrar

Subir o projeto

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python backend/app.py
```

O banco `backend/hub_arena.db` e criado sozinho na primeira execucao, com as tabelas e as procedures registradas. Painel em localhost:5000; a API responde em /api/... e pode ser aberta direto no navegador ou no Postman.

A sequencia que mostra o backend inteiro em quatro minutos

Passo	O que dizer enquanto mostra
Aceitar os termos	O aviso aparece antes de tudo e os paineis atras estao vazios: nenhuma chamada a API acontece sem consentimento, e o aceite vai para o banco.
Cadastrar um usuario	Digitar o CPF devagar para a mascara aparecer, e citar que o servidor confere os digitos verificadores e guarda so os numeros.
Criar uma reserva	Apontar o valor na tabela: ele veio do servidor. Em seguida tentar o mesmo horario na mesma quadra e mostrar o bloqueio.
Abrir os relatorios	Receita por arena (JOIN + GROUP BY), ocupacao, ranking e quadras disponiveis (NOT EXISTS). Todos sao procedure chamada pelo Repository.
Abrir o codigo	reserva_controller.py (so recebe e devolve), reserva_service.py (a regra), reserva.py (o CRUD na Model) e procedures.sql (o SQL).

17. Perguntas provaveis, e o que responder

Por que o Repository nao tem os metodos de CRUD?

Porque o CRUD simples ja esta na Model, como a arquitetura da disciplina pede. Criar um Repository so para repetir save e delete seria uma camada sem trabalho. O Repository existe para o que a Model nao faz: JOIN, GROUP BY, NOT EXISTS e as procedures.

Isso e procedure de verdade? SQLite nao tem CREATE PROCEDURE.

Nao tem mesmo, e por isso o projeto guarda cada procedure na tabela procedures do proprio banco, com o SQL parametrizado, e as executa pelo procedure_runner. O que o requisito pede — SQL fora do codigo, encapsulado no Repository, chamado por nome — esta cumprido. Controller e Service nunca executam SQL.

Por que 37 Services? Nao daria para ter um por recurso?

Daria, mas ai cada classe teria varias responsabilidades. O criterio adotado e um Service por CASO DE USO, entao cadastrar usuario e listar usuarios sao classes diferentes, com um metodo executar() cada.

Onde esta a regra de negocio?

Sempre no Service. A Controller so traduz HTTP, a Model so persiste e o Repository so consulta. O anti-overbooking e o calculo do valor estao no CriarReservaService.

Como o sistema impede duas reservas no mesmo horario?

O Service chama ReservaRepository.existe_conflito, que executa a procedure sp_conflito_reserva: ela procura reserva na mesma quadra cujo inicio seja menor que o fim pedido e cujo fim seja maior que o inicio pedido. Se achar, o Service levanta ValueError e a Controller devolve 400.

E se o usuario mandar o valor da reserva pela API?

E ignorado. O CriarReservaService nem le esse campo: ele calcula horas vezes o preco da quadra que esta no banco.

Por que a busca tem uma tela so para ela?

Porque buscar e cadastrar sao intencoes diferentes, e dividiam a mesma pagina. Separadas, a busca pode procurar em tudo — usuarios, arenas e quadras — em vez de ficar presa ao recurso daquela tela. O BuscaGlobalService coordena as tres procedures e devolve uma resposta so.

Por que as datas sao texto no banco?

O SQLite nao tem tipo de data. Guardadas em ISO, elas ordenam cronologicamente mesmo comparadas como texto, e e isso que faz o anti-overbooking e o BETWEEN dos relatorios funcionarem sem conversao.

O frontend nao deveria validar o CPF para poupar o servidor?

Ele ate ajuda com a mascara, mas validacao de verdade so vale no servidor: qualquer pessoa pode chamar a API direto, sem passar pela tela. Validar nos dois lugares e bom para a experiencia; validar so na tela e nao validar.